```
//
==================================================================
====
// UFLI COMPLETE ASSESSMENT SYSTEM WITH PROPER CALCULATIONS
// Version: 19.0 (Data & Concurrency Patched)
// Developer: Fixed and Enhanced Version
// Last Updated: 2025-10-04
//
==================================================================
====

//
==================================================================
====
// --- 1. GLOBAL CONFIGURATION ---
//
==================================================================
====
// Note: Removed 57 from this list to make 'e_e' (THESE) an assessable skill.
const REVIEW_LESSONS = new Set([5, 10, 19, 49, 53, 59, 62, 71, 76, 79, 83, 88,
92, 97, 106, 128]);

const GRADE_SKILL_CONFIG = {
  foundational: {
    name: 'Foundational Skills',
    isLessonIncluded: (n) => n >= 1 && n <= 34 && !REVIEW_LESSONS.has(n)
  },
  kindergarten_grade: {
    name: 'KG Skills',
    isLessonIncluded: (n) => {
      const ex = new Set([38, 60, 61]);
      return n >= 1 && n <= 68 && !ex.has(n) && !REVIEW_LESSONS.has(n);
    }
  },
  first_grade: {
    name: '1st Grade Skills',
    isLessonIncluded: (n) => n >= 35 && n <= 110 && !REVIEW_LESSONS.has(n)
  },
  second_grade: {
    name: '2nd Grade Skills',
    isLessonIncluded: (n) => n >= 38 && n <= 127 && !REVIEW_LESSONS.has(n)
  }
};
```

```
// Section to lesson range mapping for accurate report generation
const SECTION_LESSON_RANGES = {
  'Single Consonants & Vowels': [1, 2, 3, 4, 6, 7, 8, 9, 11, 12, 13, 14, 16, 17, 18, 20, 21,
22, 23, 24, 26, 28, 29, 30, 31, 32, 33, 34],
  'Blends': [25, 27],
  'Digraphs': [42, 43, 44, 45, 46, 47, 48, 50, 51, 52],
  'VCE (Vowel-Consonant-E)': [54, 55, 56, 57, 58, 60, 61],
  'Reading Longer Words': [63, 64, 65, 66, 67, 68],
  'Ending Spelling Patterns': [69, 70, 72, 73, 74, 75],
  'R-Controlled Vowels': [77, 78, 80, 81, 82],
  'Long Vowel Teams': [84, 85, 86, 87],
  'Other Vowel Teams': [89, 90, 91, 93, 94],
  'Diphthongs': [95, 96],
  'Silent Letters': [98],
  'Suffixes & Prefixes': [99, 100, 101, 102, 103, 104, 105],
  'Suffix Spelling Changes': [107, 108, 109, 110],
  'Low Frequency Spellings': [111, 112, 113, 114, 115, 116, 117, 118],
  'Additional Affixes': [119, 120, 121, 122, 123, 124, 125, 126, 127]
};

//
==================================================================
====
// --- 2. CORE SPREADSHEET & WEB APP FUNCTIONS ---
//
==================================================================
====
function onOpen(e) {
  try {
    const ui = SpreadsheetApp×getUi();
    const menu = ui×createMenu('UFLI Assessment Tool');
    menu.addItem('➡️ Open Assessment Form', 'showSidebar');
    menu.addItem('⚙️ Setup Required Sheets', 'setupSheets');
    menu.addSeparator();
    menu.addItem('📄 Generate Individual Report', 'generateIndividualReport');
    menu.addItem('📊 Generate Class Report', 'generateClassReport');
    menu.addItem('📈 Generate Progress Report', 'generateProgressReport');
    menu.addSeparator();
    menu.addItem('🔗 Get Web App URL', 'showWebAppUrl');
    menu.addItem('🔄 Clear Component Analysis (Keep Last 100)',
'cleanupComponentAnalysis');
    menu.addToUi();
  } catch (error) {
```

```
    console.error('Error in onOpen:', error);
  }
}

function showSidebar() {
  try {
    const html = HtmlService.createHtmlOutputFromFile('AssessmentForm')
      .setTitle('UFLI Assessment Entry Form')
      .setWidth(800);
    SpreadsheetApp.getUi().showSidebar(html);
  } catch (error) {
    SpreadsheetApp.getUi().alert('Error opening sidebar: ' + error.message);
  }
}

function doGet(e) {
  return HtmlService.createHtmlOutputFromFile('AssessmentForm')
    .setTitle('UFLI Assessment Data Entry')
    .addMetaTag('viewport', 'width=device-width, initial-scale=1');
}

function setupSheets() {
  try {
    const ss = SpreadsheetApp×getActiveSpreadsheet();
    const ui = SpreadsheetApp×getUi();

    const sheetsToCreate = {
      'Instructions': ['Step', 'Description'],
      'Configuration': ['Scorer Name'],
      'Student Roster': ['Student Name', 'Grade', 'Student ID', 'Teacher', 'Notes'],
      'Assessment Data': getAssessmentDataHeaders(),
      'Component Analysis': ['Date', 'Student', 'Word', 'Components Correct',
'Components Missed', 'Notes']
    };

    for (const sheetName in sheetsToCreate) {
      let sheet = ss×getSheetByName(sheetName);
      if (!sheet) {
        sheet = ss×insertSheet(sheetName);
      }

      if (sheet.getLastRow() > 1) {
        const response = ui×alert(
          'Sheet exists',
```

```
        `"${sheetName}" already has data. Clear it?`,
        ui.ButtonSet.YES_NO
      );
      if (response !== ui.Button.YES) continue;
    }

    sheet.clear();
    const headers = sheetsToCreate[sheetName];
    const headerRange = sheet.getRange(1, 1, 1, headers.length);
    headerRange.setValues([headers]);
    formatHeaders(headerRange);
  }

  populateInstructionsSheet();
  populateConfigurationSheet();
  applyConditionalFormatting();

  ui.alert('Setup Complete', 'All required sheets have been created/updated.',
ui.ButtonSet.OK);
 } catch (error) {
  SpreadsheetApp.getUi().alert('Setup Error: ' + error.message);
 }
}

function populateInstructionsSheet() {
 try {
  const sheet =
SpreadsheetApp.getActiveSpreadsheet().getSheetByName('Instructions');
  if (!sheet) return;

  const webAppUrl = ScriptApp.getService().getUrl();
  const instructions = [
    ['1. Add Students', 'Go to the "Student Roster" sheet and add your students.'],
    ['2. Configure Scorers', 'Go to the "Configuration" sheet and list scorers.'],
    ['3. Open the Form', 'Click "UFLI Assessment Tool" > "Open Assessment Form"
to start.'],
    ['4. Use the Web App (Optional)', 'For easier access on tablets, use the Web
App link.'],
    ['Web App Link', webAppUrl || 'Deploy your script as a Web App (Deploy > New
Deployment).'],
    ['5. Generate Reports', 'Use the menu to generate reports after submitting
assessments.'],
    ['6. Maintenance', 'Periodically clean Component Analysis using the menu
option.']
```

```
    ];

    if (sheet.getLastRow() > 1) {
      sheet.getRange(2, 1, sheet.getLastRow() - 1, 2).clearContent();
    }
    sheet.getRange(2, 1, instructions.length, 2).setValues(instructions);
    sheet.autoResizeColumns(1, 2);
  } catch (error) {
    console.error('Error populating instructions:', error);
  }
}

function populateConfigurationSheet() {
  try {
    const sheet =
SpreadsheetApp.getActiveSpreadsheet().getSheetByName('Configuration');
    if (!sheet || sheet.getLastRow() > 1) return;

    const defaultScorers = [
      ['Classroom Teacher'],
      ['Reading Specialist'],
      ['Interventionist']
    ];
    sheet.getRange(2, 1, defaultScorers.length, 1).setValues(defaultScorers);
  } catch (error) {
    console.error('Error populating configuration:', error);
  }
}

function applyConditionalFormatting() {
  try {
    const ss = SpreadsheetApp×getActiveSpreadsheet();
    const sheet = ss×getSheetByName('Assessment Data');
    if (!sheet) return;

    const greenBackground = '#d9ead3';
    const redBackground = '#f4cccc';
    const yellowBackground = '#fff2cc';
    const grayBackground = '#d9d9d9'; // Gray for Unmarked

    const headers = sheet×getRange(1, 1, 1, sheet.getLastColumn()).getValues()[0];
    const firstLessonCol = headers×indexOf('L1') + 1;
    if (firstLessonCol === 0) return;
```

```
    const lastLessonCol = headers×indexOf('L127') + 1 || sheet.getLastColumn();
    const formatRange = sheet.getRange(2, firstLessonCol, sheet.getMaxRows() - 1,
lastLessonCol - firstLessonCol + 1);

    sheet.clearConditionalFormatRules();

    const rules = [
      SpreadsheetApp.newConditionalFormatRule()
        .whenTextEqualTo('Y')
        .setBackground(greenBackground)
        .setRanges([formatRange])
        .build(),
      SpreadsheetApp.newConditionalFormatRule()
        .whenTextEqualTo('N')
        .setBackground(redBackground)
        .setRanges([formatRange])
        .build(),
      SpreadsheetApp.newConditionalFormatRule()
        .whenTextEqualTo('P')
        .setBackground(yellowBackground)
        .setRanges([formatRange])
        .build(),
      SpreadsheetApp.newConditionalFormatRule()
        .whenTextEqualTo('U')
        .setBackground(grayBackground)
        .setRanges([formatRange])
        .build()
    ];

    sheet.setConditionalFormatRules(rules);
  } catch (error) {
    console.error('Error applying conditional formatting:', error);
  }
}

function getAssessmentDataHeaders() {
  let headers = ['Date', 'Scorer', 'Student Name', 'Grade', 'Assessment Type'];

  for (let i = 1; i <= 128; i++) {
    if (!REVIEW_LESSONS.has(i)) {
      headers.push(`L${i}`);
    }
  }
```

```
  headers.push(
    'Foundational Skills %',
    'KG Skills %',
    '1st Grade Skills %',
    '2nd Grade Skills %',
    'Overall %',
    'Alphabet & Blends %',
    'Growth Since Last',
    'Notes'
  );

  return headers;
}

function formatHeaders(range) {
  range
    .setFontWeight('bold')
    .setBackground('#4285F4')
    .setFontColor('#FFFFFF')
    .setHorizontalAlignment('center')
    .setVerticalAlignment('middle')
    .setWrap(true);
}

function showWebAppUrl() {
  try {
    const url = ScriptApp×getService()×getUrl();
    const title = 'Web App URL';
    const message = url ?
      `<p>Your Web App URL is:</p>
      <input type="text" value="${url}" style="width: 95%;" readonly
onclick="this.select();">
      <p style="font-size: 12px; color: #666;">Click the URL above and press Ctrl+C
(or Cmd+C) to copy.</p>` :
      'Please deploy this script as a Web App first (Deploy > New Deployment).';

    const htmlOutput = HtmlService×createHtmlOutput(message)
      .setWidth(450)
      .setHeight(150);
    SpreadsheetApp.getUi().showModalDialog(htmlOutput, title);
  } catch (error) {
    SpreadsheetApp.getUi().alert('Error: ' + error.message);
  }
}
```

```
function cleanupComponentAnalysis() {
  try {
    const sheet =
SpreadsheetApp×getActiveSpreadsheet()×getSheetByName('Component
Analysis');
    if (!sheet || sheet.getLastRow() <= 101) {
      SpreadsheetApp.getUi().alert('No cleanup needed. Sheet has 100 or fewer
entries.');
      return;
    }

    const rowsToKeep = 100;
    const lastRow = sheet×getLastRow();
    const startRow = lastRow - rowsToKeep + 2;

    if (startRow > 2) {
      sheet.deleteRows(2, startRow - 2);
      SpreadsheetApp.getUi().alert(`Cleanup complete. Kept last ${rowsToKeep}
entries.`);
    }
  } catch (error) {
    SpreadsheetApp.getUi().alert('Cleanup Error: ' + error.message);
  }
}

//
==================================================================
====
// --- 3. DATA PROVIDERS FOR THE SIDEBAR ---
//
==================================================================
====
function getStudentList() {
  try {
    const sheet =
SpreadsheetApp×getActiveSpreadsheet()×getSheetByName('Student Roster');
    if (!sheet || sheet.getLastRow() < 2) return [];

    const data = sheet.getRange(2, 1, sheet.getLastRow() - 1, 4).getValues();
    return data
      .filter(row => row[0] && row[0].toString().trim())
      .map(row => ({
        name: row[0].toString().trim(),
```

```
        grade: row[1],
        id: row[2],
        teacher: row[3]
      }));
  } catch (error) {
    console.error('Error getting student list:', error);
    return [];
  }
}

function getScorerList() {
  try {
    const sheet =
SpreadsheetApp×getActiveSpreadsheet()×getSheetByName('Configuration');
    if (!sheet || sheet.getLastRow() < 2) return ['Default Scorer'];

    const data = sheet.getRange(2, 1, sheet.getLastRow() - 1, 1).getValues();
    const scorers = data.flat().filter(scorer => scorer && scorer.toString().trim());

    return scorers.length > 0 ? scorers : ['Default Scorer'];
  } catch (error) {
    console.error('Error getting scorer list:', error);
    return ['Default Scorer'];
  }
}

function getAssessmentSections(grade, isKindergartenEndOfYear) {
  try {
    grade = parseInt(grade);
    const allSections = JSON×parse(JSON×stringify(getAllSectionsData()));
    const sectionsToShow = [];

    const shouldSimplify = (grade === 1 || grade === 2);

    for (const key in allSections) {
      const section = allSections[key];

      if (section.gradeRanges.includes(grade)) {
        const isFoundational = (key === 'alphabet_consonants' || key === 'blends');

        if (shouldSimplify && !isFoundational) {
          section.words.forEach(word => {
            const doNotSimplify = ['COMPOUND', 'TWO_SYLLABLE',
'OPEN_SYLLABLE', 'SNAPSHOT', 'ABSENT', 'CUPID'];
```

```
          if (!doNotSimplify.includes(word.primaryComponent)) {
            word.components = [word.primaryComponent.toLowerCase()];
          }
        });
      }

      if (grade === 0) {
        if (isFoundational || (isKindergartenEndOfYear && (key === 'digraphs' || key
=== 'vce'))) {
          sectionsToShow.push(section);
        }
      } else {
        sectionsToShow.push(section);
      }
    }
  }

  return sectionsToShow;
 } catch (error) {
  console.error('Error getting assessment sections:', error);
  return [];
 }
}

//
==================================================================
====
// --- 4. CORE DATA SUBMISSION & CALCULATION ---
//
==================================================================
====
function submitAssessment(formData) {
 const lock = LockService.getScriptLock();
 lock.waitLock(30000);

 try {
  if (!formData.studentName || !formData.date || !formData.scorer) {
    return { success: false, message: 'Missing required fields (Student, Date, or
Scorer)' };
  }

  const ss = SpreadsheetApp×getActiveSpreadsheet();
  const dataSheet = ss×getSheetByName('Assessment Data');
  const componentSheet = ss×getSheetByName('Component Analysis');
```

```
  if (!dataSheet || !componentSheet) {
    return { success: false, message: 'Required sheets not found. Run Setup
first.' };
  }

  const lessonResults = {};
  for (let i = 1; i <= 128; i++) {
    if (!REVIEW_LESSONS.has(i)) {
      lessonResults[i] = '';
    }
  }

  const componentErrors = [];

  formData.sections.forEach(section => {
    section.words.forEach(wordData => {
      let componentsCorrect = [], componentsMissed = [];

      wordData.components.forEach(comp => {
        const lessonNumbers = comp.lessons;

        if (comp.correct === true) {
          componentsCorrect.push(comp.name);
          if (lessonNumbers && lessonNumbers.length > 0) {
            lessonNumbers.forEach(lesson => {
              if (lesson && !REVIEW_LESSONS.has(lesson)) {
                if (lessonResults[lesson] !== 'N') {
                  lessonResults[lesson] = 'Y';
                }
              }
            });
          }
        } else if (comp.correct === false) {
          componentsMissed.push(comp.name);
          if (lessonNumbers && lessonNumbers.length > 0) {
            lessonNumbers.forEach(lesson => {
              if (lesson && !REVIEW_LESSONS.has(lesson)) {
                lessonResults[lesson] = 'N';
              }
            });
          }
        } else {
          if (lessonNumbers && lessonNumbers.length > 0) {
```

```
          lessonNumbers.forEach(lesson => {
            if (lesson && !REVIEW_LESSONS.has(lesson)) {
              if (lessonResults[lesson] !== 'Y' && lessonResults[lesson] !== 'N') {
                lessonResults[lesson] = 'U';
              }
            }
          });
        }
      }
    });

    if (componentsMissed.length > 0) {
      componentErrors.push([
        formData.date,
        formData.studentName,
        wordData.word,
        componentsCorrect.join(', '),
        componentsMissed.join(', '),
        `Section: ${section.name || 'Unknown'}`
      ]);
    }
  });
});

if (componentErrors.length > 0) {
  const startRow = componentSheet.getLastRow() + 1;
  componentSheet.getRange(startRow, 1, componentErrors.length,
6).setValues(componentErrors);
}

const calculations = calculateAllPercentages(lessonResults,
parseInt(formData.grade), formData.studentName);

let mainRow = [
  formData.date,
  formData.scorer,
  formData.studentName,
  formData.grade,
  formData.isKindergartenEndOfYear ? 'KG End of Year' : 'Standard'
];

for (let i = 1; i <= 128; i++) {
  if (!REVIEW_LESSONS.has(i)) {
    mainRow.push(lessonResults[i] || '');
```

```
      }
    }

    mainRow.push(
      calculations.foundational,
      calculations.kindergarten,
      calculations.firstGrade,
      calculations.secondGrade,
      calculations.overall,
      calculations.alphabetBlends,
      calculations.growth,
      formData.notes || ''
    );

    dataSheet.appendRow(mainRow);

    const lastRow = dataSheet×getLastRow();
    const headers = dataSheet×getRange(1, 1, 1,
dataSheet.getLastColumn()).getValues()[0];
    const percentCols = [
      'Foundational Skills %', 'KG Skills %', '1st Grade Skills %',
      '2nd Grade Skills %', 'Overall %', 'Alphabet & Blends %', 'Growth Since Last'
    ];

    percentCols.forEach(colName => {
      const colIndex = headers.indexOf(colName) + 1;
      if (colIndex > 0) {
        dataSheet.getRange(lastRow, colIndex).setNumberFormat('0.0%');
      }
    });

    return {
      success: true,
      message: `Assessment for ${formData.studentName} submitted successfully!
`,
      summary: {
        overall: (calculations.overall * 100).toFixed(1) + '%',
        growth: calculations.growth > 0 ? '+' + (calculations.growth * 100).toFixed(1)
+ '%' : (calculations.growth * 100).toFixed(1) + '%'
      }
    };

  } catch (error) {
    console.error('Assessment submission error:', error);
```

```
    return {
      success: false,
      message: 'Submission failed: ' + error.message
    };
  } finally {
    lock.releaseLock();
  }
}

function addStudentToRoster(studentData) {
  try {
    const { name, grade } = studentData;

    if (!name || grade === "" || grade === null || grade === undefined) {
      return { success: false, message: 'Student Name and Grade are required.' };
    }

    const rosterSheet =
SpreadsheetApp×getActiveSpreadsheet()×getSheetByName('Student Roster');
    if (!rosterSheet) {
      return { success: false, message: 'Student Roster sheet not found. Run Setup
first.' };
    }

    const existingData = rosterSheet×getDataRange()×getValues();
    const nameCol = 0;
    for (let i = 1; i < existingData.length; i++) {
      if (existingData[i][nameCol] && existingData[i]
[nameCol]×toString()×toLowerCase() === name.toLowerCase()) {
        return { success: false, message: `Student "${name}" already exists.` };
      }
    }

    rosterSheet.appendRow([name, grade, '', '', 'Added ' + new
Date().toLocaleDateString()]);

    const updatedList = getStudentList();
    return {
      success: true,
      message: `${name} has been added to the roster.`,
      students: updatedList
    };

  } catch (error) {
```

```
    console.error('Error adding student:', error);
    return { success: false, message: `Error: ${error.message}` };
  }
}

function calculateAllPercentages(lessonResults, grade, studentName) {
  const calculations = {};

  const calculate = (config) => {
    let mastered = 0, tested = 0;

    for (let i = 1; i <= 128; i++) {
      if(config.isLessonIncluded(i)) {
        if (lessonResults[i] === 'Y') {
          mastered++;
          tested++;
        } else if (lessonResults[i] === 'N') {
          tested++;
        }
      }
    }

    return tested > 0 ? (mastered / tested) : 0;
  };

  calculations×foundational = calculate(GRADE_SKILL_CONFIG.foundational);
  calculations×kindergarten = (grade === 0) ?
calculate(GRADE_SKILL_CONFIG.kindergarten_grade) : 0;
  calculations×firstGrade = calculate(GRADE_SKILL_CONFIG.first_grade);
  calculations×secondGrade = calculate(GRADE_SKILL_CONFIG.second_grade);

  let totalMastered = 0, totalTested = 0;
  Object.values(lessonResults).forEach(result => {
    if (result === 'Y') {
      totalMastered++;
      totalTested++;
    } else if (result === 'N') {
      totalTested++;
    }
  });

  calculations×overall = totalTested > 0 ? (totalMastered / totalTested) : 0;

  calculations×alphabetBlends = calculate({
```

```
    isLessonIncluded: (n) => n >= 1 && n <= 34 && !REVIEW_LESSONS.has(n)
  });

  calculations×growth = calculateGrowth(studentName, calculations.overall);

  return calculations;
}

function calculateGrowth(studentName, currentOverall) {
  try {
    const dataSheet =
SpreadsheetApp×getActiveSpreadsheet()×getSheetByName('Assessment Data');
    if (!dataSheet || dataSheet.getLastRow() < 2) return 0;

    const data = dataSheet×getDataRange()×getValues();
    const headers = data[0];
    const nameIndex = headers×indexOf('Student Name');
    const overallIndex = headers×indexOf('Overall %');

    let previousOverall = null;

    for (let i = data.length - 1; i >= 1; i--) {
      if (data[i][nameIndex] === studentName && data[i][overallIndex] !== '') {
        const overallValue = data[i][overallIndex];
        if (typeof overallValue === 'number') {
          previousOverall = overallValue;
          break;
        }
      }
    }

    if (previousOverall !== null) {
      return currentOverall - previousOverall;
    }

    return 0;

  } catch (error) {
    console.error('Error calculating growth:', error);
    return 0;
  }
}

//
```

```
================================================================
====
// --- 5. REPORT GENERATION FUNCTIONS ---
//
================================================================
====
function generateIndividualReport() {
  try {
    const ui = SpreadsheetApp×getUi();
    const response = ui×prompt(
      'Generate Individual Report',
      'Enter the exact student name:',
      ui.ButtonSet.OK_CANCEL
    );

    if (response×getSelectedButton() !== ui.Button.OK || !
response.getResponseText()) {
      return;
    }

    const studentName = response×getResponseText()×trim();
    const ss = SpreadsheetApp×getActiveSpreadsheet();
    const dataSheet = ss×getSheetByName('Assessment Data');

    if (!dataSheet) {
      return ui.alert('Assessment Data sheet not found.');
    }

    const data = dataSheet×getDataRange()×getValues();
    const headers = data×shift();

    const studentAssessments = data.filter(row =>
      row[headers.indexOf('Student Name')] === studentName
    );

    if (studentAssessments.length === 0) {
      return ui.alert(`No assessments found for "${studentName}".`);
    }

    const reportSheetName = `${studentName} Report`;
    let reportSheet = ss×getSheetByName(reportSheetName);
    if (reportSheet) {
      reportSheet.clear();
    } else {
```

```
    reportSheet = ss×insertSheet(reportSheetName);
  }

  const latestAssessment = studentAssessments[studentAssessments.length - 1];
  const grade = parseInt(latestAssessment[headers.indexOf('Grade')]);
  const assessmentDate = new Date(latestAssessment[headers.indexOf('Date')]);

  let row = 1;

  reportSheet.getRange(row, 1, 1, 8).merge()
    .setValue(`${studentName} - UFLI Skills Report`)
    .setFontSize(18)
    .setFontWeight('bold')
    .setHorizontalAlignment('center');
  row += 2;

  reportSheet.getRange(row, 1, 1, 8).merge()
    .setValue(`Assessment Date: ${assessmentDate.toLocaleDateString()} | Grade:
${grade} | Assessments Completed: ${studentAssessments.length}`)
    .setHorizontalAlignment('center')
    .setFontStyle('italic');
  row += 2;

  reportSheet.getRange(row, 1).setValue('OVERALL PERFORMANCE')
    .setFontSize(14)
    .setFontWeight('bold');
  row++;

  const summaryData = [
    ['Metric', 'Current', 'Previous', 'Change'],
    ['Overall Mastery',
     latestAssessment[headers.indexOf('Overall %')],
     studentAssessments.length > 1 ?
studentAssessments[studentAssessments.length - 2][headers.indexOf('Overall
%')] : 'N/A',
     latestAssessment[headers.indexOf('Growth Since Last')]
    ],
    ['Foundational Skills',
     latestAssessment[headers.indexOf('Foundational Skills %')],
     studentAssessments.length > 1 ?
studentAssessments[studentAssessments.length - 2]
[headers.indexOf('Foundational Skills %')] : 'N/A',
     'N/A'
    ]
```

```
  ];

  if (grade === 0) {
    summaryData.push(['Kindergarten Skills',
      latestAssessment[headers.indexOf('KG Skills %')],
      studentAssessments.length > 1 ?
studentAssessments[studentAssessments.length - 2][headers.indexOf('KG Skills
%')] : 'N/A',
      'N/A'
    ]);
  } else if (grade === 1) {
    summaryData.push(['1st Grade Skills',
      latestAssessment[headers.indexOf('1st Grade Skills %')],
      studentAssessments.length > 1 ?
studentAssessments[studentAssessments.length - 2][headers.indexOf('1st Grade
Skills %')] : 'N/A',
      'N/A'
    ]);
  } else if (grade >= 2) {
    summaryData.push(['2nd Grade Skills',
      latestAssessment[headers.indexOf('2nd Grade Skills %')],
      studentAssessments.length > 1 ?
studentAssessments[studentAssessments.length - 2][headers.indexOf('2nd
Grade Skills %')] : 'N/A',
      'N/A'
    ]);
  }

  const summaryRange = reportSheet.getRange(row, 1, summaryData.length, 4);
  summaryRange.setValues(summaryData);
  reportSheet.getRange(row, 1, 1, 4).setFontWeight('bold');

  for (let i = 1; i < summaryData.length; i++) {
    reportSheet.getRange(row + i, 2).setNumberFormat('0.0%');
    if (summaryData[i][2] !== 'N/A') {
      reportSheet.getRange(row + i, 3).setNumberFormat('0.0%');
    }
    if (summaryData[i][3] !== 'N/A') {
      reportSheet.getRange(row + i, 4).setNumberFormat('+0.0%;-0.0%');
    }
  }
  row += summaryData.length + 2;

  reportSheet.getRange(row, 1).setValue('SKILL SECTION ANALYSIS')
```

```
      .setFontSize(14)
      .setFontWeight('bold');
    row++;

    const sectionResults = analyzeSectionMastery(latestAssessment, headers,
grade);
    const sectionHeaders = [['Section', 'Mastery %', 'Status', 'Lessons Assessed']];

    const sectionRows = Object.entries(sectionResults).map(([name, result]) => {
      let status = 'Needs Support';
      if (result.percentage >= 0.9) status = 'Mastered';
      else if (result.percentage >= 0.8) status = 'Proficient';
      else if (result.percentage >= 0.6) status = 'Developing';

      return [
        name,
        result.percentage,
        status,
        `${result.mastered}/${result.tested}`
      ];
    });

    if (sectionRows.length > 0) {
      reportSheet.getRange(row, 1, 1, 4).setValues(sectionHeaders);
      reportSheet.getRange(row, 1, 1, 4).setFontWeight('bold');
      row++;

      reportSheet.getRange(row, 1, sectionRows.length, 4).setValues(sectionRows);

      sectionRows.forEach((sectionRow, i) => {
        const percentCell = reportSheet.getRange(row + i, 2);
        percentCell.setNumberFormat('0.0%');

        if (sectionRow[1] >= 0.9) {
          reportSheet.getRange(row + i, 3).setBackground('#d9ead3');
        } else if (sectionRow[1] >= 0.8) {
          reportSheet.getRange(row + i, 3).setBackground('#b7e1cd');
        } else if (sectionRow[1] >= 0.6) {
          reportSheet.getRange(row + i, 3).setBackground('#fff2cc');
        } else {
          reportSheet.getRange(row + i, 3).setBackground('#f4cccc');
        }
      });
      row += sectionRows.length;
```

```
    }
    row += 2;

    const recommendations = generateRecommendations(sectionResults, grade);
    reportSheet.getRange(row, 1, 1, 8).merge()
      .setValue('INSTRUCTIONAL RECOMMENDATIONS')
      .setFontWeight('bold')
      .setFontSize(14);
    row++;

    reportSheet.getRange(row, 1, 1, 8).merge()
      .setValue(recommendations.join('\n\n'))
      .setWrap(true)
      .setVerticalAlignment('top');

    const missedLessons = getMissedLessons(latestAssessment, headers);
    if (missedLessons.length > 0 && missedLessons.length <= 10) {
      row += 3;
      reportSheet.getRange(row, 1, 1, 8).merge()
        .setValue('Priority Lessons to Review: ' + missedLessons.join(', '))
        .setFontStyle('italic');
    }

    reportSheet.autoResizeColumns(1, 8);

    ss.setActiveSheet(reportSheet);
    ui.alert(`Report for ${studentName} has been created.`);

  } catch (error) {
    SpreadsheetApp.getUi().alert('Error generating report: ' + error.message);
  }
}

function analyzeSectionMastery(assessmentRow, headers, grade) {
  const allSections = getAllSectionsData();
  const analysis = {};

  for (const key in allSections) {
    const section = allSections[key];

    if (!section.gradeRanges.includes(grade)) continue;

    const lessonRange = SECTION_LESSON_RANGES[section.name];
    if (!lessonRange || lessonRange×length === 0) continue;
```

```
    let mastered = 0, tested = 0;

    lessonRange.forEach(lessonNum => {
      const headerName = `L${lessonNum}`;
      const index = headers×indexOf(headerName);

      if (index !== -1 && (assessmentRow[index] === 'Y' || assessmentRow[index]
=== 'N')) {
        if (assessmentRow[index] === 'Y') mastered++;
        tested++;
      }
    });

    if (tested > 0) {
      analysis[section×name] = {
        mastered: mastered,
        tested: tested,
        percentage: mastered / tested
      };
    }
  }

  return analysis;
}

function getMissedLessons(assessmentRow, headers) {
  const missedLessons = [];

  for (let i = 1; i <= 128; i++) {
    if (!REVIEW_LESSONS.has(i)) {
      const headerName = `L${i}`;
      const index = headers×indexOf(headerName);

      if (index !== -1 && assessmentRow[index] === 'N') {
        missedLessons.push(`L${i}`);
      }
    }
  }

  return missedLessons;
}

function generateRecommendations(sectionResults, grade) {
```

```
  let recommendations = [];
  const critical = [], moderate = [], developing = [], strengths = [];

  for (const [name, result] of Object.entries(sectionResults)) {
    if (result.percentage >= 0.9) {
      strengths.push(name);
    } else if (result.percentage < 0.5) {
      critical.push(name);
    } else if (result.percentage < 0.7) {
      moderate.push(name);
    } else if (result.percentage < 0.9) {
      developing.push(name);
    }
  }

  if (critical.length > 0) {
    recommendations.push(`❗ IMMEDIATE INTERVENTION NEEDED: Provide
intensive, systematic instruction in ${critical.join(', ')}. Consider small group or
one-on-one support with explicit teaching and frequent progress monitoring.`);
  }
  if (moderate.length > 0) {
    recommendations.push(`⚠️ TARGETED SUPPORT: Reinforce skills in $
{moderate.join(', ')} through guided practice, scaffolded activities, and additional
repetition. Monitor progress weekly.`);
  }
  if (developing.length > 0) {
    recommendations.push(`📈 CONTINUE DEVELOPMENT: Student is
approaching mastery in ${developing.join(', ')}. Provide opportunities for
independent practice and application in context.`);
  }
  if (strengths.length > 0) {
    recommendations.push(`✅ AREAS OF STRENGTH: Student demonstrates
mastery in ${strengths.join(', ')}. Use these strengths to support learning in other
areas.`);
  }

  if (grade === 0 && critical.includes('Single Consonants & Vowels')) {
    recommendations.push(`📚 KINDERGARTEN FOCUS: Prioritize alphabet
knowledge and phonemic awareness activities. Use multisensory approaches and
frequent review.`);
  } else if (grade === 1 && (critical.includes('Digraphs') || critical.includes('VCE
(Vowel-Consonant-E)'))) {
    recommendations.push(`📖 FIRST GRADE FOCUS: Increase practice with
decodable texts focusing on targeted phonics patterns. Implement word sorts and
```

```
encoding activities.`);
  } else if (grade >= 2 && (critical.includes('Suffixes & Prefixes') ||
critical.includes('Additional Affixes'))) {
    recommendations.push(`📝 MORPHOLOGY FOCUS: Incorporate explicit
instruction in word parts and meanings. Use word building activities and
vocabulary development strategies.`);
  }

  if (recommendations×length === 0) {
    recommendations.push('Student is performing well across all assessed areas.
Continue with grade-level instruction and provide enrichment opportunities.');
  }

  return recommendations;
}

function generateClassReport() {
  try {
    const ui = SpreadsheetApp.getUi();
    const ss = SpreadsheetApp×getActiveSpreadsheet();
    const dataSheet = ss×getSheetByName('Assessment Data');
    const rosterSheet = ss×getSheetByName('Student Roster');

    if (!dataSheet || !rosterSheet) {
      return ui.alert('Error', 'Required sheets not found.', ui.ButtonSet.OK);
    }

    let reportSheet = ss×getSheetByName('Class Report');
    if (reportSheet) {
      reportSheet.clear();
    } else {
      reportSheet = ss×insertSheet('Class Report');
    }

    const rosterData = rosterSheet×getLastRow() > 1 ?
      rosterSheet.getRange(2, 1, rosterSheet.getLastRow() - 1, 2).getValues() : [];

    if (rosterData×length === 0) {
      return ui.alert('No students in roster.');
    }

    const assessmentData = dataSheet×getLastRow() > 1 ?
      dataSheet.getDataRange().getValues() : [[]];
```

```
  if (assessmentData.length < 2) {
    return ui.alert('No assessment data available.');
  }

  const headers = assessmentData×shift();
  const nameIndex = headers×indexOf('Student Name');
  const dateIndex = headers×indexOf('Date');
  const overallIndex = headers×indexOf('Overall %');
  const foundationalIndex = headers×indexOf('Foundational Skills %');
  const firstGradeIndex = headers×indexOf('1st Grade Skills %');
  const secondGradeIndex = headers×indexOf('2nd Grade Skills %');
  const growthIndex = headers×indexOf('Growth Since Last');

  const reportHeaders = [
    'Student Name', 'Grade', 'Assessments', 'Last Assessment',
    'Overall %', 'Foundational %', '1st Grade %', '2nd Grade %',
    'Growth', 'Status'
  ];

  reportSheet.getRange(1, 1, 1, reportHeaders.length).setValues([reportHeaders]);
  formatHeaders(reportSheet.getRange(1, 1, 1, reportHeaders.length));

  const assessmentMap = new Map();
  const assessmentCounts = new Map();

  assessmentData.forEach(row => {
    const studentName = row[nameIndex];
    assessmentMap.set(studentName, row);
    assessmentCounts.set(studentName, (assessmentCounts.get(studentName) ||
0) + 1);
  });

  const reportData = [];
  let classTotal = 0, classCount = 0;

  rosterData.forEach(student => {
    const studentName = student[0];
    const grade = student[1];
    const latest = assessmentMap×get(studentName);

    if (latest) {
      const overall = latest[overallIndex];
      let status = 'Needs Support';
```

```
        if (overall >= 0.9) status = 'Mastered';
        else if (overall >= 0.8) status = 'Proficient';
        else if (overall >= 0.6) status = 'Developing';

        reportData.push([
          studentName, grade, assessmentCounts.get(studentName) || 1,
          new Date(latest[dateIndex]).toLocaleDateString(), latest[overallIndex],
          latest[foundationalIndex], latest[firstGradeIndex],
          latest[secondGradeIndex], latest[growthIndex], status
        ]);

        if (typeof overall === 'number') {
          classTotal += overall;
          classCount++;
        }
      } else {
        reportData.push([
          studentName, grade, 0, 'No Assessment', '', '', '', '', '', 'Not Assessed'
        ]);
      }
    });

    if (reportData.length > 0) {
      reportSheet.getRange(2, 1, reportData.length,
reportHeaders.length).setValues(reportData);
      reportSheet.getRange(2, 5, reportData.length, 5).setNumberFormat('0.0%');

      for (let i = 0; i < reportData.length; i++) {
        const statusCell = reportSheet.getRange(i + 2, 10);
        const status = reportData[i][9];

        if (status === 'Mastered') statusCell.setBackground('#d9ead3');
        else if (status === 'Proficient') statusCell.setBackground('#b7e1cd');
        else if (status === 'Developing') statusCell.setBackground('#fff2cc');
        else if (status === 'Needs Support') statusCell.setBackground('#f4cccc');
        else statusCell.setBackground('#e0e0e0');
      }
    }

    const summaryRow = reportData×length + 3;
    reportSheet.getRange(summaryRow, 1).setValue('CLASS
AVERAGE:').setFontWeight('bold');

    if (classCount > 0) {
```

```
      reportSheet.getRange(summaryRow, 5).setValue(classTotal /
classCount).setNumberFormat('0.0%');
    }

    const distributionRow = summaryRow + 2;
    reportSheet.getRange(distributionRow, 1, 1, 4).merge()
      .setValue('PERFORMANCE DISTRIBUTION').setFontWeight('bold');

    const mastered = reportData.filter(r => r[9] === 'Mastered').length;
    const proficient = reportData.filter(r => r[9] === 'Proficient').length;
    const developing = reportData.filter(r => r[9] === 'Developing').length;
    const needsSupport = reportData.filter(r => r[9] === 'Needs Support').length;
    const notAssessed = reportData.filter(r => r[9] === 'Not Assessed').length;

    const distribution = [
      ['Status', 'Count', 'Percentage'],
      ['Mastered (90%+)', mastered, mastered / reportData.length],
      ['Proficient (80-89%)', proficient, proficient / reportData.length],
      ['Developing (60-79%)', developing, developing / reportData.length],
      ['Needs Support (<60%)', needsSupport, needsSupport / reportData.length],
      ['Not Assessed', notAssessed, notAssessed / reportData.length]
    ];

    reportSheet.getRange(distributionRow + 1, 1, distribution.length,
3).setValues(distribution);
    reportSheet.getRange(distributionRow + 1, 1, 1, 3).setFontWeight('bold');
    reportSheet.getRange(distributionRow + 2, 3, distribution.length - 1,
1).setNumberFormat('0.0%');

    reportSheet.autoResizeColumns(1, reportHeaders.length);

    ui.alert('Success', 'The "Class Report" has been generated.', ui.ButtonSet.OK);

  } catch (error) {
    SpreadsheetApp.getUi().alert('Error generating class report: ' + error.message);
  }
}

function generateProgressReport() {
  try {
    const ui = SpreadsheetApp.getUi();
    const ss = SpreadsheetApp×getActiveSpreadsheet();
    const dataSheet = ss×getSheetByName('Assessment Data');
```

```
  if (!dataSheet || dataSheet.getLastRow() < 2) {
    return ui.alert('No Data', 'Not enough assessment data for a progress report.',
ui.ButtonSet.OK);
  }

  let reportSheet = ss×getSheetByName('Progress Report');
  if (reportSheet) {
    reportSheet.clear();
  } else {
    reportSheet = ss×insertSheet('Progress Report');
  }

  const data = dataSheet×getDataRange()×getValues();
  const headers = data×shift();

  const nameIndex = headers×indexOf('Student Name');
  const dateIndex = headers×indexOf('Date');
  const overallIndex = headers×indexOf('Overall %');
  const foundationalIndex = headers×indexOf('Foundational Skills %');
  const firstGradeIndex = headers×indexOf('1st Grade Skills %');
  const secondGradeIndex = headers×indexOf('2nd Grade Skills %');

  const studentProgress = {};
  data.forEach(row => {
    const name = row[nameIndex];
    if (!studentProgress[name]) {
      studentProgress[name] = [];
    }
    studentProgress[name].push(row);
  });

  const reportHeaders = [
    'Student', 'Total Assessments', 'First Date', 'Last Date', 'Days Between',
    'Initial Overall', 'Latest Overall', 'Overall Growth',
    'Initial Found.', 'Latest Found.', 'Found. Growth',
    'Initial 1st Gr', 'Latest 1st Gr', '1st Gr Growth',
    'Initial 2nd Gr', 'Latest 2nd Gr', '2nd Gr Growth',
    'Growth Rate', 'Trend'
  ];

  reportSheet.getRange(1, 1, 1, reportHeaders.length).setValues([reportHeaders]);
  formatHeaders(reportSheet.getRange(1, 1, 1, reportHeaders.length));

  const progressData = [];
```

```
  for (const name in studentProgress) {
    const assessments = studentProgress[name];

    if (assessments.length > 1) {
      const first = assessments[0];
      const last = assessments[assessments.length - 1];

      const firstDate = new Date(first[dateIndex]);
      const lastDate = new Date(last[dateIndex]);
      const daysBetween = Math×round((lastDate - firstDate) / (1000 × 60 × 60 ×
24));

      const initialOverall = first[overallIndex];
      const latestOverall = last[overallIndex];
      const overallGrowth = latestOverall - initialOverall;

      const initialFoundational = first[foundationalIndex];
      const latestFoundational = last[foundationalIndex];
      const foundationalGrowth = latestFoundational - initialFoundational;

      const initialFirstGrade = first[firstGradeIndex];
      const latestFirstGrade = last[firstGradeIndex];
      const firstGradeGrowth = latestFirstGrade - initialFirstGrade;

      const initialSecondGrade = first[secondGradeIndex];
      const latestSecondGrade = last[secondGradeIndex];
      const secondGradeGrowth = latestSecondGrade - initialSecondGrade;

      const growthRate = daysBetween > 0 ? (overallGrowth / daysBetween) * 30 :
0;

      let trend = 'Stable';
      if (overallGrowth > 0.1) trend = 'Improving';
      else if (overallGrowth > 0.05) trend = 'Slight Improvement';
      else if (overallGrowth < -0.05) trend = 'Declining';

      progressData.push([
        name, assessments.length, firstDate.toLocaleDateString(),
lastDate.toLocaleDateString(),
        daysBetween, initialOverall, latestOverall, overallGrowth,
        initialFoundational, latestFoundational, foundationalGrowth,
        initialFirstGrade, latestFirstGrade, firstGradeGrowth,
        initialSecondGrade, latestSecondGrade, secondGradeGrowth,
```

```
        growthRate, trend
      ]);
    }
  }

  progressData.sort((a, b) => b[7] - a[7]);

  if (progressData.length > 0) {
    reportSheet.getRange(2, 1, progressData.length,
reportHeaders.length).setValues(progressData);

    reportSheet.getRange(2, 6, progressData.length, 3).setNumberFormat('0.0%');
    reportSheet.getRange(2, 9, progressData.length, 3).setNumberFormat('0.0%');
    reportSheet.getRange(2, 12, progressData.length,
3).setNumberFormat('0.0%');
    reportSheet.getRange(2, 15, progressData.length,
3).setNumberFormat('0.0%');
    reportSheet.getRange(2, 18, progressData.length,
1).setNumberFormat('0.0%');

    for (let i = 0; i < progressData.length; i++) {
      const trendCell = reportSheet.getRange(i + 2, 19);
      const trend = progressData[i][18];

      if (trend === 'Improving') trendCell.setBackground('#d9ead3');
      else if (trend === 'Slight Improvement') trendCell.setBackground('#b7e1cd');
      else if (trend === 'Stable') trendCell.setBackground('#fff2cc');
      else if (trend === 'Declining') trendCell.setBackground('#f4cccc');

      const overallGrowthCell = reportSheet.getRange(i + 2, 8);
      if (progressData[i][7] > 0.05) overallGrowthCell.setBackground('#d9ead3');
      else if (progressData[i][7] > 0) overallGrowthCell.setBackground('#b7e1cd');
      else if (progressData[i][7] < -0.05)
overallGrowthCell.setBackground('#f4cccc');
      else overallGrowthCell.setBackground('#fff2cc');
    }
  } else {
    reportSheet.getRange(2, 1).setValue('No students have multiple assessments
yet.');
  }

  reportSheet.autoResizeColumns(1, reportHeaders.length);

  ui.alert('Success', 'The "Progress Report" has been generated.',
```

```
ui.ButtonSet.OK);

  } catch (error) {
    SpreadsheetApp.getUi().alert('Error generating progress report: ' +
error.message);
  }
}

//
==================================================================
====
// --- 6. COMPLETE ASSESSMENT DATA (SOURCE OF TRUTH) ---
//
==================================================================
====
function getAllSectionsData() {
  return {
    alphabet_consonants: {
      name: 'Single Consonants & Vowels', gradeRanges: [0, 1, 2],
      words: [
        { word: 'SAM', number: 1, primaryComponent: 'S', components: ['s', 'a', 'm'],
lessons: { 's': [3], 'a': [1], 'm': [2] } },
        { word: 'PIN', number: 2, primaryComponent: 'P', components: ['p', 'i', 'n'],
lessons: { 'p': [6], 'i': [8], 'n': [9] } },
        { word: 'COD', number: 3, primaryComponent: 'C', components: ['c', 'o', 'd'],
lessons: { 'c': [14], 'o': [12], 'd': [13] } },
        { word: 'FAN', number: 4, primaryComponent: 'F', components: ['f', 'an'],
lessons: { 'f': [7], 'an': [11] } },
        { word: 'BUG', number: 5, primaryComponent: 'B', components: ['b', 'u', 'g'],
lessons: { 'b': [17], 'u': [15], 'g': [16] } },
        { word: 'BEGS', number: 6, primaryComponent: 'S', components: ['b', 'e', 'g',
's'], lessons: { 'b': [17], 'e': [18], 'g': [16], 's': [20] } },
        { word: 'KITS', number: 7, primaryComponent: 'K', components: ['k', 'i', 't', 's'],
lessons: { 'k': [22], 'i': [8], 't': [4], 's': [20] } },
        { word: 'HAS', number: 8, primaryComponent: 'H', components: ['h', 'a', 's'],
lessons: { 'h': [23], 'a': [1], 's': [3] } },
        { word: 'QUIZ', number: 9, primaryComponent: 'QU', components: ['qu', 'i',
'z'], lessons: { 'qu': [32], 'i': [8], 'z': [34] } },
        { word: 'FOX', number: 10, primaryComponent: 'X', components: ['f', 'o', 'x'],
lessons: { 'f': [7], 'o': [12], 'x': [31] } },
        { word: 'LET', number: 11, primaryComponent: 'L', components: ['l', 'e', 't'],
lessons: { 'l': [26], 'e': [18], 't': [4] } },
        { word: 'YUM', number: 12, primaryComponent: 'Y', components: ['y', 'u', 'm'],
lessons: { 'y': [30], 'u': [15], 'm': [2] } },
```

```
      { word: 'WIG', number: 13, primaryComponent: 'W', components: ['w', 'i', 'g'],
lessons: { 'w': [28], 'i': [8], 'g': [16] } },
      { word: 'RUN', number: 14, primaryComponent: 'R', components: ['r', 'u', 'n'],
lessons: { 'r': [24], 'u': [15], 'n': [9] } },
      { word: 'VET', number: 15, primaryComponent: 'V', components: ['v', 'e', 't'],
lessons: { 'v': [33], 'e': [18], 't': [4] } },
      { word: 'JOG', number: 16, primaryComponent: 'J', components: ['j', 'o', 'g'],
lessons: { 'j': [29], 'o': [12], 'g': [16] } }
    ]
  },
  blends: {
    name: 'Blends', gradeRanges: [0, 1, 2],
    words: [
      { word: 'BRICK', number: 1, primaryComponent: 'BR', components: ['br'],
lessons: { 'br': [25] } },
      { word: 'CREEK', number: 2, primaryComponent: 'CR', components: ['cr'],
lessons: { 'cr': [25] } },
      { word: 'DRIVE', number: 3, primaryComponent: 'DR', components: ['dr'],
lessons: { 'dr': [25] } },
      { word: 'FROG', number: 4, primaryComponent: 'FR', components: ['fr'],
lessons: { 'fr': [25] } },
      { word: 'GREEN', number: 5, primaryComponent: 'GR', components: ['gr'],
lessons: { 'gr': [25] } },
      { word: 'PRETTY', number: 6, primaryComponent: 'PR', components: ['pr'],
lessons: { 'pr': [25] } },
      { word: 'TRUCK', number: 7, primaryComponent: 'TR', components: ['tr'],
lessons: { 'tr': [25] } },
      { word: 'BLACK', number: 8, primaryComponent: 'BL', components: ['bl'],
lessons: { 'bl': [27] } },
      { word: 'CLIP', number: 9, primaryComponent: 'CL', components: ['cl'],
lessons: { 'cl': [27] } },
      { word: 'FLOP', number: 10, primaryComponent: 'FL', components: ['fl'],
lessons: { 'fl': [27] } },
      { word: 'GLUM', number: 11, primaryComponent: 'GL', components: ['gl'],
lessons: { 'gl': [27] } },
      { word: 'PLUG', number: 12, primaryComponent: 'PL', components: ['pl'],
lessons: { 'pl': [27] } },
      { word: 'SLOW', number: 13, primaryComponent: 'SL', components: ['sl'],
lessons: { 'sl': [27] } }
    ]
  },
  digraphs: {
    name: 'Digraphs', gradeRanges: [0, 1, 2, 3, 4, 5, 6, 7, 8],
    words: [
```

{ word: 'JAZZ', number: 1, primaryComponent: 'ZZ', components: ['j', 'a', 'zz','z'], lessons: { 'j': [29], 'a': [1], 'z' : [34], 'zz': [42] } },
{ word: 'DRESS', number: 2, primaryComponent: 'SS', components: ['dr', 'e', 'ss'], lessons: { 'dr': [25], 'e': [18], 'ss': [42] } },
{ word: 'SPILL', number: 3, primaryComponent: 'LL', components: ['i', 'll'], lessons: { 'i': [8], 'll': [42] } },
{ word: 'OFF', number: 4, primaryComponent: 'FF', components: ['o', 'ff'], lessons: { 'o': [12], 'ff': [42] } },
{ word: 'ALL', number: 5, primaryComponent: 'ALL', components: ['all'], lessons: { 'all': [43] } },
{ word: 'LUNCH', number: 6, primaryComponent: 'CH', components: ['l', 'u', 'ch'], lessons: { 'l': [26], 'u': [15], 'ch': [48] } },
{ word: 'THIN', number: 7, primaryComponent: 'TH', components: ['th','n'], lessons: { 'th': [46], 'n' :[9]} },
{ word: 'THEM', number: 8, primaryComponent: 'TH', components: ['th'], lessons: { 'th': [47] } },
{ word: 'SH (in shock)', number: 9, primaryComponent: 'SH', components: ['sh'], lessons: { 'sh': [45] } },
{ word: 'CK (in shock)', number: 10, primaryComponent: 'CK', components: ['ck'], lessons: { 'ck': [44] } },

{ word: 'WHEN', number: 11, primaryComponent: 'WH', components: ['wh'], lessons: { 'wh': [50]} },
{ word: 'GRAPH', number: 12, primaryComponent: 'PH', components: ['ph'], lessons: { 'ph': [50] } },
{ word: 'PING', number: 13, primaryComponent: 'NG', components: ['p', 'ng'], lessons: { 'p': [6], 'ng': [51] } },
{ word: 'FANG', number: 14, primaryComponent: 'NG', components: ['f', 'ng'], lessons: { 'f': [7], 'ng': [51] } },
{ word: 'RINK', number: 15, primaryComponent: 'NK', components: ['r', 'nk'], lessons: { 'r': [24], 'nk': [52] } }
]
},
vce: {
name: 'VCE (Vowel-Consonant-E)', gradeRanges: [0, 1, 2, 3, 4, 5, 6, 7, 8],
words: [
{ word: 'PHONE', number: 1, primaryComponent: 'O_E', components: ['o_e'], lessons: { 'o_e': [56] } },
{ word: 'GRAPE', number: 2, primaryComponent: 'A_E', components: ['a_e'], lessons: { 'a_e': [54] } },
{ word: 'QUITE', number: 3, primaryComponent: 'I_E', components: ['qu', 'i_e'], lessons: { 'qu': [32], 'i_e': [55] } },

```
      { word: 'THESE', number: 4, primaryComponent: 'E_E', components: ['e_e'],
lessons: { 'e_e': [57] } },
      { word: 'MUTE', number: 5, primaryComponent: 'U_E', components: ['m',
'u_e'], lessons: { 'm': [2], 'u_e': [58] } },
      { word: 'DUNE', number: 6, primaryComponent: 'U_E', components: ['d',
'u_e'], lessons: { 'd': [13], 'u_e': [58] } },
      { word: 'ICE', number: 7, primaryComponent: 'CE', components: ['ce'],
lessons: { 'ce': [60] } },
      { word: 'PAGE', number: 8, primaryComponent: 'GE', components: ['ge'],
lessons: { 'ge': [61] } }
    ]
  },
  reading_longer_words: {
    name: 'Reading Longer Words', gradeRanges: [1, 2, 3, 4, 5, 6, 7, 8],
    words: [
      { word: 'BENCHES', number: 1, primaryComponent: 'ES', components: ['es'],
lessons: { 'es': [63] } },
      { word: 'YELLED', number: 2, primaryComponent: 'ED', components: ['ed'],
lessons: { 'ed': [64] } },
      { word: 'JUMPED', number: 3, primaryComponent: 'ED', components: ['ed'],
lessons: { 'ed': [64] } },
      { word: 'BLASTED', number: 4, primaryComponent: 'ED', components: ['bl',
'ed'], lessons: { 'bl': [27], 'ed': [64] } },
      { word: 'BOXING', number: 5, primaryComponent: 'ING', components: ['x',
'ing'], lessons: { 'x': [31], 'ing': [65] } },
      { word: 'SNAPSHOT', number: 6, primaryComponent: 'SNAPSHOT',
components: ['snap', 'shot'], lessons: { 'snap': [66], 'shot': [66] } },
      { word: 'ABSENT', number: 7, primaryComponent: 'ABSENT', components:
['ab', 'sent'], lessons: { 'ab': [67], 'sent': [67] } },
      { word: 'CUPID', number: 8, primaryComponent: 'CUPID', components: ['cu',
'pid'], lessons: { 'cu': [68], 'pid': [68] } }
    ]
  },
  ending_spelling_patterns: {
    name: 'Ending Spelling Patterns', gradeRanges: [1, 2, 3, 4, 5, 6, 7, 8],
    words: [
      { word: 'SWITCH', number: 1, primaryComponent: 'TCH', components: ['w',
'tch'], lessons: { 'w': [28], 'tch': [69] } },
      { word: 'PLEDGE', number: 2, primaryComponent: 'DGE', components: ['pl',
'dge'], lessons: { 'pl': [27], 'dge': [70] } },
      { word: 'CHILD', number: 3, primaryComponent: 'ILD', components: ['ild'],
lessons: { 'ild': [72] } },
      { word: 'BLIND', number: 4, primaryComponent: 'IND', components: ['bl',
'ind'], lessons: { 'bl': [27], 'ind': [72] } },
```

```
      { word: 'VOLT', number: 5, primaryComponent: 'OLT', components: ['v', 'olt'],
lessons: { 'v': [33], 'olt': [72] } },
      { word: 'POST', number: 6, primaryComponent: 'OST', components: ['p', 'ost'],
lessons: { 'p': [6], 'ost': [72] } },
      { word: 'GOLD', number: 7, primaryComponent: 'OLD', components: ['g', 'old'],
lessons: { 'g': [16], 'old': [72] } },
      { word: 'TRY', number: 8, primaryComponent: 'Y_AS_I', components: ['tr', 'y'],
lessons: { 'tr': [25], 'y': [74] } },
      { word: 'TABLE', number: 9, primaryComponent: 'LE', components: ['t', 'a',
'ble'], lessons: { 't': [4], 'a': [1], 'ble': [75] } },
      { word: 'SAMPLE', number: 10, primaryComponent: 'LE', components: ['s',
'am', 'ple'], lessons: { 's': [3], 'am': [11], 'ple': [75] } },
      { word: 'CANDY', number: 11, primaryComponent: 'Y_AS_E', components: ['y'],
lessons: { 'y': [73],'y' : [30] } }
    ]
  },
  r_controlled_vowels: {
    name: 'R-Controlled Vowels', gradeRanges: [1, 2, 3, 4, 5, 6, 7, 8],
    words: [
      { word: 'CART', number: 1, primaryComponent: 'AR', components: ['c', 'ar'],
lessons: { 'c': [14], 'ar': [77] } },
      { word: 'PORT', number: 2, primaryComponent: 'OR', components: ['or'],
lessons: { 'or': [78] } },
      { word: 'CLERK', number: 3, primaryComponent: 'ER', components: ['cl', 'er'],
lessons: { 'cl': [27], 'er': [80] } },
      { word: 'BIRDS', number: 4, primaryComponent: 'IR', components: ['b', 'ir', 's'],
lessons: { 'b': [17], 'ir': [81], 's': [21] } },
      { word: 'HURT', number: 5, primaryComponent: 'UR', components: ['h', 'ur'],
lessons: { 'h': [23], 'ur': [82] } }
    ]
  },
  long_vowel_teams: {
    name: 'Long Vowel Teams', gradeRanges: [1, 2, 3, 4, 5, 6, 7, 8],
    words: [
      { word: 'PAINT', number: 1, primaryComponent: 'AI', components: ['ai'],
lessons: { 'ai': [84] } },
      { word: 'SWAY', number: 2, primaryComponent: 'AY', components: ['ay'],
lessons: { 'ay': [84] } },
      { word: 'AI (in railway)', number: 3, primaryComponent: 'AI', components: ['ai'],
lessons: { 'ai': [84] } },
      { word: 'AY (in railway)', number: 4, primaryComponent: 'AY', components:
['ay'], lessons: { 'ay': [84] } },
      { word: 'GREET', number: 5, primaryComponent: 'EE', components: ['gr', 'ee'],
lessons: { 'gr': [25], 'ee': [85] } },
```

{ word: 'BLEACH', number: 6, primaryComponent: 'EA', components: ['ea'], lessons: { 'ea': [85] } },
{ word: 'KIDNEY', number: 7, primaryComponent: 'EY', components: ['k', 'ey'], lessons: { 'k': [22], 'ey': [84] } },
{ word: 'THROAT', number: 8, primaryComponent: 'OA', components: ['oa'], lessons: { 'oa': [86] } },
{ word: 'GROW', number: 9, primaryComponent: 'OW', components: ['ow'], lessons: { 'ow': [86] } },
{ word: 'TOE', number: 10, primaryComponent: 'OE', components: ['oe'], lessons: { 'oe': [86] } },
{ word: 'LIFEBOAT', number: 11, primaryComponent: 'OA', components: ['oa'], lessons: { 'oa': [86] } },
{ word: 'LIE', number: 12, primaryComponent: 'IE', components: ['ie'], lessons: { 'ie': [87] } },
{ word: 'BRIGHT', number: 13, primaryComponent: 'IGH', components: ['br', 'igh'], lessons: { 'br': [25], 'igh': [87] } },
{ word: 'INSIGHT', number: 14, primaryComponent: 'IGH', components: ['igh'], lessons: { 'igh': [87] } }
]
},
other_vowel_teams: {
name: 'Other Vowel Teams', gradeRanges: [1, 2, 3, 4, 5, 6, 7, 8],
words: [
{ word: 'COOK', number: 1, primaryComponent: 'OO', components: ['oo'], lessons: { 'oo': [89] } },
{ word: 'PUSH', number: 2, primaryComponent: 'U', components: ['u'], lessons: { 'u': [89] } },
{ word: 'MOON', number: 3, primaryComponent: 'OO', components: ['oo'], lessons: { 'oo': [90] } },
{ word: 'CHEW', number: 4, primaryComponent: 'EW', components: ['ew'], lessons: { 'ew': [91] } },
{ word: 'FRUIT', number: 5, primaryComponent: 'UI', components: ['ui'], lessons: { 'ui': [91] } },
{ word: 'GLUE', number: 6, primaryComponent: 'UE', components: ['gl', 'ue'], lessons: { 'gl': [27], 'ue': [91] } },
{ word: 'CLAW', number: 7, primaryComponent: 'AW', components: ['aw'], lessons: { 'aw': [93] } },
{ word: 'GAUZE', number: 8, primaryComponent: 'AU', components: ['au'], lessons: { 'au': [93] } },
{ word: 'CAUGHT', number: 9, primaryComponent: 'AUGH', components: ['augh'], lessons: { 'augh': [93] } },
{ word: 'DREAD', number: 10, primaryComponent: 'EA', components: ['dr', 'ea'], lessons: { 'dr': [25], 'ea': [94] } },
{ word: 'SWAN', number: 11, primaryComponent: 'A', components: ['a', 's'],

lessons: { 'a': [94] , 's' : [20]} }
      ]
    },
    diphthongs: {
      name: 'Diphthongs', gradeRanges: [1, 2, 3, 4, 5, 6, 7, 8],
      words: [
        { word: 'JOIN', number: 1, primaryComponent: 'OI', components: ['oi'],
lessons: { 'oi': [95] } },
        { word: 'JOY', number: 2, primaryComponent: 'OY', components: ['oy'],
lessons: { 'oy': [95] } },
        { word: 'SHOUT', number: 3, primaryComponent: 'OU', components: ['ou'],
lessons: { 'ou': [96] } },
        { word: 'PLOW', number: 4, primaryComponent: 'OW', components: ['ow'],
lessons: { 'ow': [96] } }
      ]
    },
    silent_letters: {
      name: 'Silent Letters', gradeRanges: [1, 2, 3, 4, 5, 6, 7, 8],
      words: [
        { word: 'KNEE', number: 1, primaryComponent: 'KN', components: ['kn'],
lessons: { 'kn': [98] } },
        { word: 'WRITE', number: 2, primaryComponent: 'WR', components: ['wr'],
lessons: { 'wr': [98] } },
        { word: 'CLIMB', number: 3, primaryComponent: 'MB', components: ['mb'],
lessons: { 'mb': [98] } }
      ]
    },
    suffixes_prefixes: {
      name: 'Suffixes & Prefixes', gradeRanges: [1, 2, 3, 4, 5, 6, 7, 8],
      words: [
        { word: 'CATS', number: 1, primaryComponent: '-S', components: ['s'],
lessons: { 's': [99] } },
        { word: 'BUNCHES', number: 2, primaryComponent: '-ES', components: ['es'],
lessons: { 'es': [99] } },
        { word: 'FASTER', number: 3, primaryComponent: '-ER', components: ['er'],
lessons: { 'er': [100] } },
        { word: 'COLDEST', number: 4, primaryComponent: '-EST', components:
['est'], lessons: { 'est': [100] } },
        { word: 'SADLY', number: 5, primaryComponent: '-LY', components: ['ly'],
lessons: { 'ly': [101] } },
        { word: 'UNPACK', number: 6, primaryComponent: 'UN-', components: ['un'],
lessons: { 'un': [103] } },
        { word: 'ENDLESS', number: 7, primaryComponent: '-LESS', components:
['less'], lessons: { 'less': [102] } },

{ word: 'HELPFUL', number: 8, primaryComponent: '-FUL', components: ['ful'], lessons: { 'ful': [102] } },
{ word: 'UNKIND', number: 9, primaryComponent: 'UN-', components: ['un'], lessons: { 'un': [103] } },
{ word: 'PREMADE', number: 10, primaryComponent: 'PRE-', components: ['pre'], lessons: { 'pre': [104] } },
{ word: 'RERUN', number: 11, primaryComponent: 'RE-', components: ['re'], lessons: { 're': [104] } },
{ word: 'DISLIKE', number: 12, primaryComponent: 'DIS-', components: ['dis'], lessons: { 'dis': [105] } }
]
},
suffix_spelling_changes: {
name: 'Suffix Spelling Changes', gradeRanges: [1, 2, 3, 4, 5, 6, 7, 8],
words: [
{ word: 'SLAPPED', number: 1, primaryComponent: 'PP and ED', components: ['pp', 'ed'], lessons: { 'pp': [107], 'ed': [107] } },
{ word: 'DROPPING', number: 2, primaryComponent: 'PP and ING', components: ['pp', 'ing'], lessons: { 'pp': [107], 'ing': [107] } },
{ word: 'THINNER', number: 3, primaryComponent: 'NN and ER', components: ['nn', 'er'], lessons: { 'nn': [108], 'er': [108] } },
{ word: 'MADDEST', number: 4, primaryComponent: 'DD and EST', components: ['dd', 'est'], lessons: { 'dd': [108], 'est': [108] } },
{ word: 'BRAVER', number: 5, primaryComponent: 'ADD -ER', components: ['er'], lessons: { 'er': [100] } },
{ word: 'CUTEST', number: 6, primaryComponent: 'ADD -EST', components: ['est'], lessons: { 'est': [100] } },
{ word: 'FADED', number: 7, primaryComponent: 'ADD -ED', components: ['ed'], lessons: { 'ed': [107] } },
{ word: 'HOPING', number: 8, primaryComponent: 'DROP E', components: ['ing'], lessons: { 'ing': [109] } },
{ word: 'CARRIED', number: 9, primaryComponent: 'Y>IED', components: ['ied'], lessons: { 'ied': [110] } },
{ word: 'CRIES', number: 10, primaryComponent: 'Y>IES', components: ['y>ies'], lessons: { 'y>ies': [110] } },
{ word: 'BABIES', number: 11, primaryComponent: 'Y>IES', components: ['y>ies'], lessons: { 'y>ies': [110] } },
{ word: 'HANDIER', number: 12, primaryComponent: 'Y>IER', components: ['y>ier'], lessons: { 'y>ier': [110] } },
{ word: 'LAZIEST', number: 13, primaryComponent: 'Y>IEST', components: ['y>iest'], lessons: { 'y>iest': [110] } }
]
},
low_frequency_spellings: {

name: 'Low Frequency Spellings', gradeRanges: [1, 2, 3, 4, 5, 6, 7, 8],
words: [
{ word: 'BEGGAR', number: 1, primaryComponent: 'AR', components: ['ar'], lessons: { 'ar': [111] } },
{ word: 'ACTOR', number: 2, primaryComponent: 'OR', components: ['or'], lessons: { 'or': [111] } },
{ word: 'HAIR', number: 3, primaryComponent: 'AIR', components: ['air'], lessons: { 'air': [112] } },
{ word: 'DARE', number: 4, primaryComponent: 'ARE', components: ['are'], lessons: { 'are': [112] } },
{ word: 'PEAR', number: 5, primaryComponent: 'EAR', components: ['ear'], lessons: { 'ear': [112] } },
{ word: 'CLEAR', number: 6, primaryComponent: 'EAR', components: ['ear'], lessons: { 'ear': [113] } },
{ word: 'VEIN', number: 7, primaryComponent: 'EI', components: ['ei'], lessons: { 'ei': [114] } },
{ word: 'OBEY', number: 8, primaryComponent: 'EY', components: ['ey'], lessons: { 'ey': [114] } },
{ word: 'EIGHT', number: 9, primaryComponent: 'EIGH', components: ['eigh'], lessons: { 'eigh': [114] } },
{ word: 'STEAK', number: 10, primaryComponent: 'EA', components: ['ea'], lessons: { 'ea': [114] } },
{ word: 'FEW', number: 11, primaryComponent: 'EW', components: ['ew'], lessons: { 'ew': [115] } },
{ word: 'FEUD', number: 12, primaryComponent: 'EU', components: ['eu'], lessons: { 'eu': [115] } },
{ word: 'VALUE', number: 13, primaryComponent: 'UE', components: ['ue'], lessons: { 'ue': [115] } },
{ word: 'OUGHT', number: 14, primaryComponent: 'OUGH', components: ['ough'], lessons: { 'ough': [116] } },
{ word: 'DOUGH', number: 15, primaryComponent: 'OUGH', components: ['ough'], lessons: { 'ough': [116] } },
{ word: 'CENTER', number: 16, primaryComponent: 'C', components: ['c'], lessons: { 'c': [117] } },
{ word: 'AGENT', number: 17, primaryComponent: 'G', components: ['g'], lessons: { 'g': [117] } },
{ word: 'GNAT', number: 18, primaryComponent: 'GN', components: ['gn'], lessons: { 'gn': [118] } },
{ word: 'GHOST', number: 19, primaryComponent: 'GH', components: ['gh'], lessons: { 'gh': [118] } },
{ word: 'CHEF', number: 20, primaryComponent: 'CH', components: ['ch'], lessons: { 'ch': [118] } },
{ word: 'ECHO', number: 21, primaryComponent: 'CH', components: ['ch'], lessons: { 'ch': [118] } },

```
      { word: 'CASTLE', number: 22, primaryComponent: 'T', components: ['t'],
lessons: { 't': [118] } }
     ]
   },
   additional_affixes: {
     name: 'Additional Affixes', gradeRanges: [1, 2, 3, 4, 5, 6, 7, 8],
     words: [
      { word: 'INVASION', number: 1, primaryComponent: '-SION', components:
['sion'], lessons: { 'sion': [119] } },
      { word: 'OPTION', number: 2, primaryComponent: '-TION', components:
['tion'], lessons: { 'tion': [119] } },
      { word: 'NATURE', number: 3, primaryComponent: '-TURE', components:
['ture'], lessons: { 'ture': [120] } },
      { word: 'FIGHTER', number: 4, primaryComponent: '-ER', components: ['er'],
lessons: { 'er': [121] } },
      { word: 'DIRECTOR', number: 5, primaryComponent: '-OR', components: ['or'],
lessons: { 'or': [121] } },
      { word: 'ARTIST', number: 6, primaryComponent: '-IST', components: ['ist'],
lessons: { 'ist': [121] } },
      { word: 'DARKISH', number: 7, primaryComponent: '-ISH', components: ['ish'],
lessons: { 'ish': [122] } },
      { word: 'SANDY', number: 8, primaryComponent: '-Y', components: ['y'],
lessons: { 'y': [123] } },
      { word: 'REDNESS', number: 9, primaryComponent: '-NESS', components:
['ness'], lessons: { 'ness': [124] } },
      { word: 'PAVEMENT', number: 10, primaryComponent: '-MENT', components:
['ment'], lessons: { 'ment': [125] } },
      { word: 'FIXABLE', number: 11, primaryComponent: '-ABLE', components:
['able'], lessons: { 'able': [126] } },
      { word: 'FLEXIBLE', number: 12, primaryComponent: '-IBLE', components:
['ible'], lessons: { 'ible': [126] } },
      { word: 'UNICORN', number: 13, primaryComponent: 'UNI-', components:
['uni'], lessons: { 'uni': [127] } },
      { word: 'BISECT', number: 14, primaryComponent: 'BI-', components: ['bi'],
lessons: { 'bi': [127] } },
      { word: 'TRIGRAPH', number: 15, primaryComponent: 'TRI-', components:
['tri'], lessons: { 'tri': [127] } }
     ]
   }
 };
}

//
==========================================================
```

```
====
// --- END OF SCRIPT ---
//
================================================================
====
```